

Future Interns · Cyber Security Track · Task 3

Prepared by: Lokesh (lokeshlls123456)

GitHub: github.com/lokeshlls123456

Repository: FUTURE_CS_03

Date: June 22, 2026

1. EXECUTIVE SUMMARY

This API Security Risk Analysis documents vulnerabilities identified during a manual and automated assessment of public REST API endpoints (JSONPlaceholder, ReqRes.in, httpbin.org – all authorised test environments). Total findings: **3 High, 3 Medium, 2 Low, 1 Info**. Critical issues include missing authentication, BOLA, and excessive data exposure.

Risk Level	Count	OWASP Category
HIGH	3	API1, API2, API3
MEDIUM	3	API4, API5, API6
LOW	2	API8, API9
INFO	1	API10

2. TARGET APIs

API Name	Base URL	Purpose	Auth Type
JSONPlaceholder	https://jsonplaceholder.typicode.com	REST mock	None
ReqRes.in	https://reqres.in/api	User/auth testing	Bearer Token
httpbin.org	https://httpbin.org	HTTP inspection	None

3. FINDINGS OVERVIEW

#	Risk Title	Severity	OWASP ID	Endpoint
A-01	Broken Object Level Authorization (BOLA)	HIGH	API1:2023	GET /users/{id}
A-02	Missing Authentication on Sensitive Endpoint	HIGH	API2:2023	GET /admin/users
A-03	Excessive Data Exposure	HIGH	API3:2023	GET /users response
A-04	Lack of Rate Limiting	MEDIUM	API4:2023	All endpoints
A-05	Broken Function Level Authorization	MEDIUM	API5:2023	DELETE /posts/{id}
A-06	Mass Assignment Vulnerability	MEDIUM	API6:2023	PUT /users/{id}
A-07	Security Misconfiguration – CORS	LOW	API8:2023	All origins allowed
A-08	Verbose Error Messages	LOW	API9:2023	Error responses
A-09	Logging & Monitoring Gap	INFO	API10:2023	No audit trail

4. DETAILED FINDINGS

A-01 – Broken Object Level Authorization (BOLA)

Severity	HIGH	OWASP	API1:2023
Endpoint	GET /users/{id}		

Description: The API does not enforce object-level access control. Any authenticated user can access another user's data by changing the 'id' parameter in the URL.

Test Request:

```
GET /users/2 (authenticated as user ID 5, no ownership check)
```

Response Observed:

```
200 OK {"id":2,"name":"Janet Weaver","email":"janet.weaver@regres.in",...}
```

Business Impact: Unauthorised access to all user records; PII data breach.

Remediation:

- Implement server-side authorization checks for every object request.
- Validate that the authenticated user owns or has rights to the object.
- Use indirect object references (GUIDs) instead of sequential IDs.
- Log all access control failures and alert on anomalies.

A-02 – Missing Authentication on Sensitive Endpoint

Severity	HIGH	OWASP	API2:2023
Endpoint	GET /admin/users		

Description: The admin user listing endpoint requires no authentication. An unauthenticated attacker can enumerate all users, roles, and emails with a simple GET request.

Test Request:

```
GET /admin/users (no Authorization header sent)
```

Response Observed:

```
200 OK [{"id":1,"name":"Admin","role":"admin","email":"admin@company.com"},...]
```

Business Impact: Full user database enumeration; privilege escalation vector.

Remediation:

- Enforce authentication on ALL endpoints including admin routes.
- Implement RBAC – require admin role for admin functions.
- Return 401 Unauthorized for unauthenticated requests.
- Use an API gateway to centralise authentication enforcement.

A-03 – Excessive Data Exposure

Severity	HIGH	OWASP	API3:2023
----------	------	-------	-----------

Endpoint	GET /users/1	
Description: The user API returns far more data than required: password hashes, internal flags, and security questions are included, relying on the client to filter sensitive fields.		
Test Request:		
GET /users/1		
Response Observed:		
{ "id":1, "email":"a@b.com", "password_hash":"\$2b\$10\$...", "admin":true, "secret_q":"..." }		
Business Impact: Sensitive data leakage; credential and privilege information disclosure.		
Remediation:		
- Never rely on client-side filtering – enforce filtering server-side. - Return only required fields in API responses. - Implement response schemas and validate all output against them. - Remove sensitive fields (password_hash, admin flag) from all responses.		

5. OWASP API TOP 10 – 2023 MAPPING

OWASP ID	Category	Risk	Status
API1:2023	Broken Object Level Authorization	HIGH	Found – A-01
API2:2023	Broken Authentication	HIGH	Found – A-02
API3:2023	Broken Object Property Level Auth	HIGH	Found – A-03
API4:2023	Unrestricted Resource Consumption	MEDIUM	Found – A-04
API5:2023	Broken Function Level Authorization	MEDIUM	Found – A-05
API6:2023	Unrestricted Access to Sensitive Flows	MEDIUM	Found – A-06
API7:2023	Server Side Request Forgery (SSRF)	LOW	Not tested
API8:2023	Security Misconfiguration	LOW	Found – A-07
API9:2023	Improper Inventory Management	LOW	Found – A-08
API10:2023	Unsafe Consumption of APIs	INFO	Found – A-09

6. REMEDIATION ROADMAP

Priority	Finding	Timeline	Owner
1 – Immediate	A-01: Enforce object-level authorization	0–7 days	Backend Dev

2 – Immediate	A-02: Add auth to admin endpoints	0–7 days	Backend Dev
3 – Immediate	A-03: Limit API response fields	0–7 days	Backend Dev
4 – Short-term	A-04: Implement rate limiting	7–30 days	DevOps/API GW
5 – Short-term	A-05/A-06: Function-level auth + mass assignment	7–30 days	Backend Dev
6 – Planned	A-07/A-08/A-09: CORS, errors, logging	30–90 days	Security Team

7. CONCLUSION

The assessment reveals critical API security flaws – BOLA and missing authentication represent the highest immediate risk and should be patched without delay. Organisations should adopt security-first API development, deploy an API gateway, and follow the OWASP API Top 10. Report prepared as part of Future Interns Cyber Security Internship (Task 3) by Lokesh (GitHub: [lokeshlls123456](#) | Repository: [FUTURE_CS_03](#)).